

PaperProof Protocol Yellow Paper

A Source-Derived Specification of the Sui-Native Artifact Protocol

PaperProof Labs
paperproof.labs@gmail.com
<https://paperproof.site/>
<https://github.com/PaperProofLabs>

Draft v0.2

June 2026

Copyright and Use Notice

Copyright © 2026 PaperProof Labs. All rights reserved.

This yellow paper is published as a source-available protocol specification for review, citation, education, auditing, SDK development, indexing, and integration with the official PaperProof Protocol deployments. Permission is granted to read, download, print, quote limited portions, cite, and use this document for the purpose of understanding, evaluating, or integrating with PaperProof.

No permission is granted by this document to use the PaperProof name, PaperProof Labs name, logos, trademarks, deployment identity, official manifests, or other branding in a confusing or misleading manner. No permission is granted to claim that an independent implementation, fork, derivative protocol, or compatible system is the official PaperProof Protocol unless expressly authorized by PaperProof Labs or by the applicable future governance process.

This document does not relicense the PaperProof smart contracts, SDKs, website, logos, trademarks, or other project materials. The smart-contract source code remains governed by its own license notices, including `LicenseRef-PaperProof-Source-Available` where applicable. Implementers are responsible for complying with those licenses and with applicable law.

This document describes protocol concepts, object relationships, event semantics, and deployment expectations. It is not a patent license, trademark license, endorsement, warranty, or grant of official status. PaperProof Labs reserves all rights not expressly granted here.

Abstract

PaperProof is a Sui-native protocol for durable, verifiable, and community-governed digital artifacts. This yellow paper specifies the current contract-level protocol as implemented by the PaperProof mainnet packages. It describes the trust roots, deployment manifest, object inventory, artifact type registry, artifact series model, typed version records, Walrus content references, comments tree, likes book, fee manager, governance vault, proposals, voting, claims, events, validation rules, error codes, indexing rules, upgrade boundaries, and SDK-facing contracts.

This document is intentionally narrower than a whitepaper and stricter than an application guide. It does not claim that PaperProof proves truth, originality, academic quality, license compliance, or social value. It specifies what the protocol recognizes, what the contracts store, what events can be treated as canonical under the active deployment, and how third-party applications and indexers should avoid common integration errors.

Contents

1	Status and Scope	4
1.1	Normative Language	4
1.2	What This Yellow Paper Specifies	4
1.3	What This Yellow Paper Does Not Specify	5
2	System Overview	5
3	Deployment Manifest	6
3.1	Required Manifest Fields	6
3.2	Drift Checks	6
4	Protocol Object Inventory	6
5	Artifact Types	8
6	Artifact Series and Version Records	9
6.1	Series Identity	9
6.2	Common Version Header	9
6.3	Metadata Extensions	10
7	Publication and Version State Transitions	10
7.1	Publication	10
7.2	Adding Versions	11
7.3	Ownership Transfer	11
8	Walrus Content References	12
9	Comments Tree and Likes Book	12
9.1	Official Binding	12
9.2	Comment Tree Status and Comment Status	12
9.3	On-Chain and Blob Comments	13
9.4	Comment Moderation	13
9.5	Likes	13
10	Protocol-Native Prompt Registry	13
10.1	Prompt Registration Key	13
10.2	Prompt Governance Boundary	14
11	Copilot Memory Registry	14
11.1	Provider Policy	14
11.2	Memory Entry State	14
11.3	Memory Operations	15
12	Fee Model	15
13	Governance	15
13.1	Governance Vault	15
13.2	Governance Config and Proposals	16
13.3	Governance Action Types	16
13.4	Voting and Claims	17
13.5	Execution	17

14 Events	17
14.1 Publishing Events	17
14.2 Comment and Like Events	18
14.3 Governance Events	18
14.4 Prompt Registry Events	18
14.5 Memory Registry Events	19
15 Canonical Indexing Rules	19
15.1 Canonical Acceptance	19
15.2 Derived API Views	20
15.3 Empty State vs Failed Query	20
15.4 Protocol Facts vs Interface Policy	20
16 Validation Rules	21
17 SDK-Facing Contracts	21
17.1 Reference Indexer API Expectations	22
18 Upgrade and Compatibility	22
19 Security Boundaries	22
20 Implementation Evidence and Verification Surfaces	23
A Mainnet Deployment Snapshot	25
B Abort Code Summary	25
B.1 Publishing Abort Codes	25
B.2 Validation Abort Codes	26
B.3 Comments Abort Codes	26
B.4 Prompt Registry Abort Codes	27
B.5 Memory Registry Abort Codes	27
B.6 Governance Voting Abort Codes	27
C References	28

1 Status and Scope

This document is derived from the PaperProof Move source code and deployment documentation in the contract repository. The relevant source modules are:

- `paperproof_publishing::publishing`
- `paperproof_publishing::artifact_types`
- `paperproof_publishing::validation`
- `paperproof_comments::comments`
- `paperproof_governance::governance`
- `paperproof_governance::governance_voting`
- `paperproof_prompt_registry::prompt_registry`
- `paperproof_memory_registry::memory_registry`

The default network is mainnet. Network-specific package and object bindings are not hardcoded into the abstract protocol. They are resolved through deployment manifests. The current mainnet manifest is summarized in [Appendix A](#). Future testnet manifests should follow the same schema.

1.1 Normative Language

The words “MUST”, “MUST NOT”, “SHOULD”, and “MAY” are used in their ordinary specification sense. A frontend, SDK, or indexer that ignores a MUST-level condition may show incorrect protocol state, accept non-canonical events, or build transactions that are expected to fail.

1.2 What This Yellow Paper Specifies

This yellow paper specifies:

- protocol objects and their relationships;
- built-in artifact type constants and typed version records;
- publication and add-version state transitions;
- comment tree, blob comment, like, and unlike semantics;
- fee collection and fee level semantics;
- governance proposal, voting, execution, expiration, and claim semantics;
- event interpretation and canonical indexing rules;
- validation limits and abort-code tables;
- deployment manifest and drift-check expectations;
- SDK-facing behavior that follows from contract semantics.

1.3 What This Yellow Paper Does Not Specify

This document does not define a social feed, recommendation algorithm, full-text search system, moderation policy, peer-review process, airdrop rule, ranking algorithm, or token valuation model. It also does not make the official frontend or SDKs the source of protocol truth. The contracts and active deployment manifest define protocol bindings; SDKs and frontends are reference integration layers.

2 System Overview

PaperProof separates large content from compact authoritative state. WALRUS stores large blobs such as PDFs, datasets, software archives, long comments, and static frontend assets. SUI stores official protocol objects, typed commitments, object bindings, fees, governance state, and canonical events. The protocol identity of an artifact is not merely a content hash or storage URL. It is a series object under an official root, with typed versions, official comments tree, official likes book, and events accepted under the active deployment.

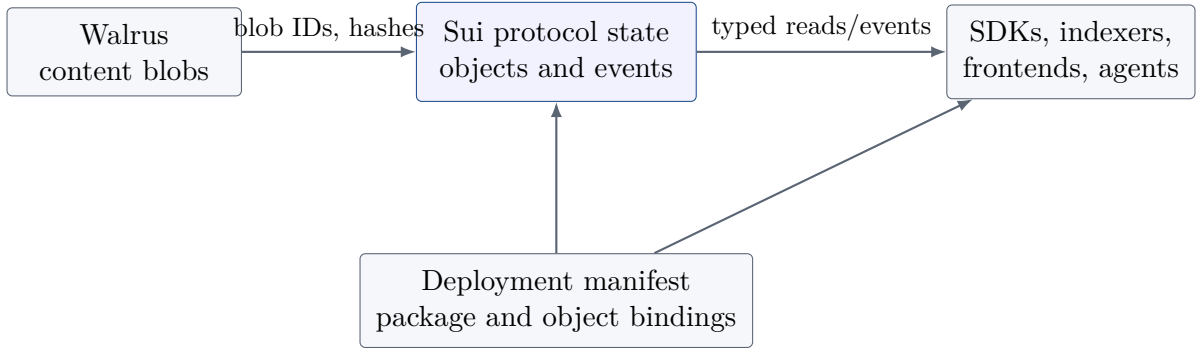


Figure 1: PaperProof separates content storage, protocol state, and integration surfaces.

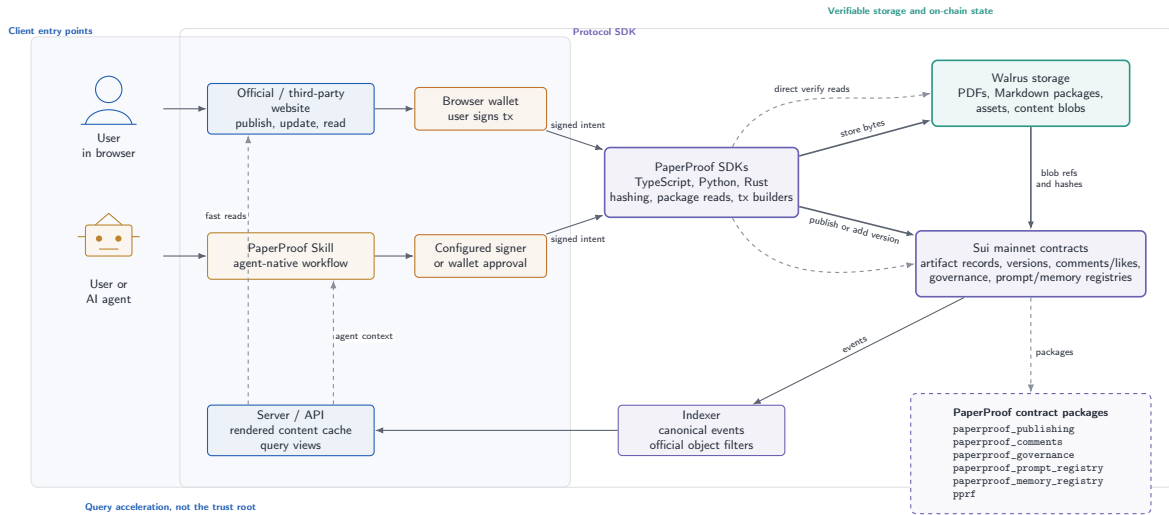


Figure 2: Application-level architecture and flow. The official or third-party website and the PaperProof Skill submit and query artifact workflows through the same Sui mainnet contract packages, Walrus content layer, server-side indexer, and SDK surfaces.

3 Deployment Manifest

A PaperProof client **MUST** resolve network-specific package IDs, object IDs, coin types, and protocol version metadata from a deployment manifest. A deployment manifest is a JSON document with a schema version and a **deployment** object. It **SHOULD** also include package history so indexers can classify old events without treating them as current canonical events.

3.1 Required Manifest Fields

At minimum, a manifest **MUST** provide:

- network name, for example **mainnet** or future **testnet**;
- protocol version label;
- package IDs for **pprf**, **publishing**, **comments**, and **governance**;
- package IDs for optional official capability registries, including prompt and memory registries when enabled by an official app build;
- object IDs for the root, type registry, fee manager, governance vault, governance config, and **SUI Clock**;
- object IDs for official shared capability registries where deployed;
- coin types for **PPRF**, **SUI**, and **WAL** where relevant;
- an update timestamp and minimum recommended SDK version.

3.2 Drift Checks

SDKs and indexers **SHOULD** fail closed when a manifest does not match chain state. A drift check **SHOULD** verify that:

- the root's governance vault, fee manager, and type registry IDs match the manifest;
- the governance vault's registry ID matches the official registry;
- the governance vault's governance config ID matches the manifest;
- the fee manager's registry ID matches the official registry;
- the comments trees and likes books accepted by the application are bound to official series objects;
- the prompt registry and memory registry, if used, bind to the same root ID and governance vault ID as the active manifest;
- event package IDs match the current package for current-state updates, while package history is used only for migration-aware historical views.

4 Protocol Object Inventory

Table 1 lists the primary protocol objects. Getter functions exist for many fields; this paper lists semantic object roles rather than every read function.

Table 1: Primary protocol object inventory.

Object	Role
PaperProofRoot	Top-level trust anchor. Stores root version, paused flag, official governance vault ID, fee manager ID, type registry ID, comments tree factory capability, and governance action executor capability.
TypeRegistry	Registry of supported artifact types. Stores registry version, registry ID, and a table from artifact type code to TypeInfo .
TypeInfo	Per-type metadata: artifact type, type index object ID, enabled flag, schema version, minimum protocol version, creation timestamp, and update timestamp.
TypeIndex	Per-type object used to support artifact code mapping and type-scoped discovery. It stores version, registry ID, and artifact type.
ArtifactSeries	Stable identity of a continuing artifact. Stores type, code, controller binding and compatibility owner mirror, version IDs, current version, official comments tree, official likes book, status, UI status, metadata extensions, and timestamps.
ControllerNFT	Transferable control asset for one artifact series. Stores series binding, artifact type, control-record binding, issuance metadata, and transfer-relevant state. Holding the NFT is the canonical series-control authority.
ArtifactControlRecord	Canonical control-binding object. Stores the series ID, controller NFT ID, current controller mirror, compatibility owner mirror, official comments tree binding, authority mode, and timestamps used by contracts, SDKs, and indexers to resolve series authority.
CommonArtifactHeader	Shared version header for all typed version records. Stores series ID, type, version number, previous version ID, author, content hash, Walrus references, content type, metadata extensions, status, and timestamp.
PreprintVersionRecord	Typed version record for preprints. Adds title, abstract, authors, keywords, field, license, and page count.
BlogPostVersionRecord	Typed version record for blog posts. Adds title, summary, tags, and language.
TechnicalReportVersionRecord	Typed version record for technical reports. Adds title, abstract, authors, organization, report number, keywords, and license.
DatasetVersionRecord	Typed version record for datasets. Adds title, description, format, file count, size in bytes, license, and keywords.
SoftwareReleaseVersionRecord	Typed version record for software releases. Adds project name, version name, source hash, package hash, changelog, license, and repository URL.
GenericFileVersionRecord	Typed version record for generic files. Adds title, description, filename, file size, and license.
CommentsTree	Official comment space for a series. Stores target series, target artifact type, root comment, nodes table, controller-aligned moderation owner mirror, status, fee manager binding, max inline size, depth limit, and likes book ID.

Object	Role
CommentNode	Single comment node. Stores parent, author, depth, content mode, inline content or blob references, children count, timestamps, and status.
LikesBook	Official like state for a series. Stores comments tree ID, target series ID, target type, like count, and address-to-like table.
GovernanceVault	Authority and treasury boundary. Stores governance authority, upgrade authority, active operator, pending operator transfer, fee recipient, direct authority mode, and related capability state.
FeeManager	Official fee configuration. Stores registry ID and fee levels for comments and artifact types.
GovernanceConfig	Proposal system configuration. Stores total supply, proposer threshold, duration, next proposal ID, pause state, active proposal ID, proposal ID mapping, and enabled actions.
Proposal	Governance proposal object. Stores proposal type, action type, title, description, payload fields, votes, locked balances, epoch window, status, execution flag, and vote records.
PromptRegistry	Official route-to-prompt registry for app prompts. Stores root binding, governance vault binding, and prompt registrations keyed by app ID and route ID. Updates require the active governance operator.
PromptRegistration	Copyable prompt binding record. Stores app ID, route ID, prompt artifact series ID, optional pinned version ID, latest-version policy, schema version, update epoch, and updater.
MemoryRegistry	Official Copilot memory capability registry. Stores root binding, governance vault binding, provider policies, and active memory entry IDs by wallet owner and app ID.
ProviderPolicy	Memory provider policy. Stores provider name, enabled flag, accepted schema-version range, update epoch, and updater.
MemoryEntry	Per-wallet, per-app memory capability object. Stores owner, app ID, memory ID, provider, MemWal account ID, namespace root, descriptor artifact code, descriptor series/version policy, schema version, availability, owner-enabled state, owner-deleted tombstone, and timestamps.

5 Artifact Types

Artifact types are protocol-recognized constants. They are not arbitrary labels. Adding a new protocol-recognized type is a contract and SDK change followed by governance activation.

The artifact code function constructs a human-readable code:

`PaperProof-{type}-{epoch6}-{series_id_hex12}`

The artifact code is useful for display and search, but the authoritative identity remains the series object ID plus official root and registry binding.

Table 2: Built-in artifact types.

Value	Name	Primary typed record
1	preprint	PreprintVersionRecord
2	blog_post	BlogPostVersionRecord
3	technical_report	TechnicalReportVersionRecord
4	dataset	DatasetVersionRecord
5	software_release	SoftwareReleaseVersionRecord
6	generic_file	GenericFileVersionRecord

6 Artifact Series and Version Records

6.1 Series Identity

ArtifactSeries is the protocol identity of a continuing work. A series **MUST** have exactly one artifact type and one artifact code. A series **MAY** have many versions up to `MAX_VERSIONS_PER_SERIES = 168`. The current implementation stores version IDs in a vector and tracks the current version number and current version object ID.

Every controlled series **MUST** also have exactly one canonical controller asset represented by a **ControllerNFT** and exactly one **ArtifactControlRecord** that binds the series, the controller NFT, and the official comments tree. The controller NFT is the source of write authority for series-scoped operations. The control record exists so contracts, SDKs, and indexers can resolve control state deterministically without inferring it only from frontend conventions.

Control authority is distinct from content license and historical authorship. Transferring the controller NFT changes who may publish new versions, update series-level metadata, and exercise artifact-scoped moderation authority. It does not rewrite prior versions, change the recorded author of a version, or replace the license terms committed by earlier version records.

Series status values are:

Table 3: Series status values.

Value	Meaning
0	active
1	locked
2	hidden

Only active series are intended to receive normal new versions. Hidden status is a protocol field that official applications may interpret as non-display state. Status changes emit **ArtifactStatusChangedEvent**.

6.2 Common Version Header

Every typed version record contains **CommonArtifactHeader**. This header is the shared commitment layer. It binds the version to:

- series ID;
- artifact type;
- version number;
- previous version ID if present;

- author address;
- content hash;
- Walrus blob ID;
- Walrus blob object ID;
- content type;
- bounded metadata extensions;
- version status;
- chain timestamp in milliseconds.

The current version status constant is `VERSION_STATUS_VALID = 0`. Version records are shared objects once created and are not mutated by later version additions.

6.3 Metadata Extensions

Series and version metadata extensions are vectors of `MetadataAttribute`. Limits are:

Table 4: Metadata extension limits.	
Limit	Value
<code>MAX_METADATA_ATTRIBUTES</code>	4
<code>MAX_METADATA_KEY_BYTES</code>	64
<code>MAX_METADATA_VALUE_BYTES</code>	511

Keys MUST be non-empty and unique within the vector. Metadata extensions are display and indexing aids; they MUST NOT be used as the sole authority for core protocol semantics.

7 Publication and Version State Transitions

7.1 Publication

Publication functions are type-specific:

- `publish_preprint`
- `publish_blog_post`
- `publish_technical_report`
- `publish_dataset`
- `publish_software_release`
- `publish_generic_file`

Each publication call MUST provide the official root, type registry, governance vault, fee manager, type-specific metadata, common content fields, optional series and version metadata extensions, optional SUI payment, SUI Clock, and transaction context.

Publication performs the following logical steps:

1. validate type-specific metadata and common content fields;

2. verify root, registry, vault, and fee manager bindings;
3. verify the protocol is not paused;
4. verify the artifact type is supported and enabled;
5. collect the configured artifact fee, if any;
6. create an **ArtifactSeries**;
7. create a typed version record with version number 1;
8. mint the series **ControllerNFT** and create the **ArtifactControlRecord**;
9. create the official comments tree and likes book;
10. bind controller authority to the official comments tree moderation boundary;
11. share the series, comments tree, likes book, and typed version record, and return the controller NFT to the publisher wallet;
12. emit **ArtifactPublishedEvent** and related creation events.

7.2 Adding Versions

Add-version functions are also type-specific:

- `add_preprint_version`
- `add_blog_post_version`
- `add_technical_report_version`
- `add_dataset_version`
- `add_software_release_version`
- `add_generic_file_version`

An add-version transaction **MUST** be authorized by the holder of the series **ControllerNFT** or by a protocol-recognized execution path that presents controller authority for the same series. It **MUST** match the series artifact type, **MUST** keep the series within the maximum version count, **MUST** use official root/registry/vault/fee manager bindings, and **MUST** produce a typed version record for the same artifact type. It emits **ArtifactVersionAddedEvent**.

7.3 Ownership Transfer

Artifact control is transferred by transferring the series **ControllerNFT**. The receiving address becomes the canonical controller and therefore gains the right to add versions, update series metadata, and administer artifact-scoped moderation policy. Implementations **MAY** maintain series-level and comments-tree owner mirrors for compatibility and display, but such mirrors are not the canonical source of control authority.

Controller transfer **MUST NOT** mutate prior version records, prior version authors, committed content hashes, historical Walrus references, or prior license selections. It changes future operational authority over the series; it does not rewrite the historical evidence already committed by earlier versions.

Indexers and applications **SHOULD** resolve current series control from the **ControllerNFT** plus the bound **ArtifactControlRecord**, not from stale display mirrors or cached frontend

ownership assumptions. If an application shows a compatibility owner mirror, that mirror SHOULD be treated as advisory display state rather than as the final source of protocol authority.

Marketplace listing, escrow, auction, lending, or bundled sale of a **ControllerNFT** MAY be implemented by external Sui infrastructure. Such commercial flows are outside PaperProof's protocol scope until a valid NFT transfer settles on chain. Once settlement occurs, the receiving holder becomes the canonical controller under the same artifact series ID and control-record binding.

8 Walrus Content References

PaperProof does not store large content bytes in SUI objects. Instead, version records store content references:

- **content_hash**: application-selected content digest string;
- **walrus_blob_id**: Walrus blob identifier;
- **walrus_blob_object_id**: Walrus blob object identifier;
- **content_type**: media type or application content type string.

The contract validates that these fields are non-empty and within bounded lengths. SDKs SHOULD verify content locally before publication when possible. The protocol records the commitment; it does not download or validate blob bytes inside Move.

9 Comments Tree and Likes Book

9.1 Official Binding

Every published artifact series receives an official **CommentsTree** and **LikesBook**. A frontend or indexer MUST NOT accept an arbitrary comments tree as official only because it has the expected Move type. It MUST verify that the series object stores the tree ID and likes book ID, and that the tree/book fields bind to the same registry, series, and artifact type.

9.2 Comment Tree Status and Comment Status

Table 5: Comment tree and comment status values.

Domain	Value	Meaning
Tree	0	open
Tree	1	locked
Tree	2	archived
Comment	0	active
Comment	1	hidden
Comment	2	deleted

Only open trees accept new comments. A parent comment MUST exist and MUST be active. The root comment has ID 0 and cannot be mutated as an ordinary user comment.

An application MAY use tree status to create non-commentable official surfaces. For example, Docs articles can be published as ordinary artifact series while their official comments tree is set to **locked** or **archived**. The frontend SHOULD hide comment controls for such surfaces, but the protocol reason new comments fail is the tree status check.

9.3 On-Chain and Blob Comments

Comment content modes are:

Table 6: Comment content modes.

Value	Meaning
1	inline on-chain content
2	blob-backed content

Inline comments **MUST** be non-empty and no larger than the tree’s configured maximum inline-comment byte limit, whose default is 512 bytes. Blob comments **MUST** include non-empty blob ID and blob digest and **MAY** include a content preview. Blob comment limits are:

Table 7: Blob comment limits.

Limit	Value
MAX_BLOB_ID_BYTES	128
MAX_BLOB_DIGEST_BYTES	128
MAX_CONTENT_PREVIEW_BYTES	256
DEFAULT_MAX_COMMENT_DEPTH	64

9.4 Comment Moderation

The tree owner may set tree status and comment status. A comment author may delete their own comment, but cannot restore hidden comments or modify the root comment. Deleted comments are final for status purposes. Status changes emit `TreeStatusChangeEvent` or `CommentStatusChangeEvent`.

9.5 Likes

`like_paper` and `unlike_paper` operate on the official `LikesBook`. The caller **MUST** provide a PPRF coin proof whose value is at least `MIN_PPRF_FOR_LIKE`. A like is proof of holding, not a burn or stake. The likes book prevents duplicate likes by the same address and emits `PaperLikedEvent` or `PaperUnlikedEvent`.

10 Protocol-Native Prompt Registry

The prompt registry maps official application routes to versioned PaperProof prompt artifacts. Prompt bytes are not stored in the registry. They are stored as ordinary `generic_file` artifacts whose version records commit to Walrus references. The registry only selects which artifact series and version policy an official app route should use.

10.1 Prompt Registration Key

A prompt registration is keyed by `app_id + ":" + route_id`. In the official app, examples include `paperproof-app:copilot/global` and `paperproof-app:copilot/memory`. The registry stores:

- `app_id` and `route_id`;
- `series_id` of the prompt artifact;

- optional `pinned_version_id`;
- `use_latest`, which selects the current version of the series when true;
- `schema_version`;
- update epoch and updater address.

10.2 Prompt Governance Boundary

`create_registry` and `register_prompt` MUST be called by the active operator of the root-bound governance vault. The registry MUST bind to the same root ID and governance vault ID expected by the active deployment manifest. This is intentionally lighter than proposal-based automatic execution: it gives official operators a governed capability boundary without requiring every prompt update to be a full token vote.

Clients MUST resolve prompt content in two steps. First, read the official prompt registration. Second, read the referenced PaperProof artifact series/version through normal artifact rules. If `use_latest` is false, `pinned_version_id` MUST be present. If chain resolution, Walrus fetch, or manifest drift checks fail, an app MAY fall back to bundled prompts, but it MUST not claim those bundled prompts are the current chain-selected prompt.

11 Copilot Memory Registry

The memory registry is a governed discovery and availability layer for wallet-linked Copilot memory. It does not store private memory bodies. Memory bodies are handled by an external provider such as MemWal, which may in turn use Walrus storage. PaperProof stores only metadata needed for official discovery, version policy, and availability checks.

11.1 Provider Policy

Before a provider is considered usable, the registry MUST contain a `ProviderPolicy`. A policy records provider name, enabled flag, minimum and maximum accepted schema versions, update epoch, and updater. Setting provider policy requires the active governance operator.

11.2 Memory Entry State

Each `MemoryEntry` is a shared object. The official app model is at most one active entry per wallet owner and app ID. Deleted entries are tombstoned, not physically erased, and the active-entry mapping is released so the owner can create a new entry later.

Table 8: Memory entry usability flags.

Field	Meaning
<code>available</code>	Operator/governance availability flag. If false, the official Copilot MUST NOT use the entry.
<code>owner_enabled</code>	User-controlled enablement flag for the chain entry.
<code>owner_deleted</code>	Tombstone flag. Deleted entries are not usable and do not block later registration.
<code>use_latest</code>	Version policy for the descriptor artifact. If false, a pinned version ID is required.

A memory entry is usable only when all of the following hold: it belongs to the official registry, `available` is true, `owner_enabled` is true, `owner_deleted` is false, the provider policy

exists, the provider policy is enabled, and the entry schema version is within the provider policy range.

11.3 Memory Operations

The create-entry function creates a shared entry and records its ID in the owner/app active-entry table. The call MUST reject creation if an active entry already exists for the same owner and app ID. The update-pointer function lets the owner update the provider account pointer and namespace root. The disable-entry function lets the owner set `owner_enabled` to false. The delete-entry function releases the active-entry mapping, sets `owner_enabled` to false, sets `owner_deleted` to true, sets `available` to false, and records a deletion epoch. It does not delete external MemWal or Walrus data.

`set_memory_availability` and `set_memory_version_policy` require the active governance operator and the official registry/vault binding. They let the official app disable a specific memory capability or change the descriptor series/version policy used to explain the memory schema.

12 Fee Model

`FeeManager` stores fee levels rather than arbitrary per-call fee values. The current fee level constants are:

Table 9: Fee level constants.

Value	Name
0	free
1	micro
2	low
3	standard
4	high

Comments use fee key `FEE_KEY_COMMENTS = 0`. Artifact publishing and versioning use artifact-type fee levels. Fees are collected through the official governance vault and fee manager. A fake fee manager with the same field names MUST NOT be accepted by clients.

13 Governance

13.1 Governance Vault

`GovernanceVault` is the authority boundary for governance and operational roles. It stores:

- governance config ID;
- governance authority;
- upgrade authority;
- active operator and operator epoch;
- pending operator transfer state;
- fee recipient;
- direct authority mode;

- permanent direct-authority-disable flag.

Direct authority modes are:

Table 10: Direct authority modes.

Value	Meaning
0	full
1	emergency
2	read-only
3	disabled

Permanent disablement is one-way. Once direct authority is permanently disabled, the protocol cannot return to a stronger direct-authority mode.

13.2 Governance Config and Proposals

GovernanceConfig stores proposal parameters, action enablement, active proposal tracking, and proposal ID mapping. **Proposal** stores proposal metadata, payload, vote totals, locked PPRF balances, epoch window, status, and per-address vote records.

Proposal types are:

Table 11: Proposal types.

Value	Meaning
1	executable
2	signal

Proposal status values are:

Table 12: Proposal status values.

Value	Meaning
1	active
2	passed
3	rejected
4	executed
5	expired

13.3 Governance Action Types

Table 13: Governance action type constants.

Value	Action
2	set comments fee level
3	set fee recipient
4	nominate operator
5	set proposal creation paused
6	set proposer threshold

Value	Action
7	set upgrade authority
8	set proposal duration epochs
9	set artifact type enabled
10	set artifact fee level
11	activate artifact type
12	set governance action enabled
13	set direct authority mode
14	cancel operator transfer
15	set governance authority
101	signal replace operator
102	signal feature direction
103	signal policy position

13.4 Voting and Claims

Proposal creation requires a proposer stake of PPRF at least equal to the configured proposer threshold. The proposer stake is counted as a yes vote and locked in the proposal. Additional voters call `vote_yes` or `vote_no` with PPRF coins. Minimum vote stake is 100_000_000_000 base units, documented in code as 100 PPRF.

After voting ends, `finalize_proposal` determines whether the proposal passed or failed. `resolve_proposal_early` may finalize early only when the outcome is determinable. After a proposal leaves active status, voters can call `claim_locked_tokens` to recover locked funds. A voter can claim only once because the vote record is removed during claim.

13.5 Execution

Executable proposals can be executed after passing and before execution expiry. The general `execute_proposal` path handles actions that mutate the governance vault or config directly, such as fee recipient, operator nomination, pause flag, proposer threshold, upgrade authority, proposal duration, governance action enablement, direct authority mode, and governance authority.

Some actions are intentionally consumed through a `GovernanceActionTicket` and then applied by target modules. This pattern is used for artifact type enabled, artifact fee level, artifact type activation, and comments fee level. The ticket prevents a generic proposal from becoming an unrestricted capability.

14 Events

Events are evidence for off-chain systems, but they are not self-authenticating. A canonical event MUST come from the active package and MUST match the official root, registry, vault, fee manager, series, comments tree, or likes book binding expected for the event type.

14.1 Publishing Events

Publishing emits:

- `PaperProofRootCreatedEvent`
- `TypeRegistryCreatedEvent`
- `TypeIndexCreatedEvent`

- `ArtifactPublishedEvent`
- `ArtifactVersionAddedEvent`
- `ArtifactStatusChangedEvent`
- `ArtifactSeriesMetadataUpdatedEvent`
- `ArtifactTypeStatusChangedEvent`
- `ProtocolPausedChangedEvent`

14.2 Comment and Like Events

Comment and like events are:

- `TreeCreatedEvent`
- `CommentAddedEvent`
- `TreeStatusChangedEvent`
- `CommentStatusChangedEvent`
- `TreeOwnerTransferredEvent`
- `CommentsTreeMigratedEvent`
- `PaperLikedEvent`
- `PaperUnlikedEvent`

14.3 Governance Events

Governance emits:

- creation events: `GovernanceVaultCreatedEvent`, `FeeManagerCreatedEvent`, and `GovernanceConfigCreatedEvent`;
- proposal events: `ProposalCreatedEvent`, `VoteCastEvent`, `ProposalFinalizedEvent`, `ProposalExecutedEvent`, `ProposalExpiredEvent`, and `VoteClaimedEvent`;
- parameter events: `ProposalCreationPausedChangedEvent`, `ProposerThresholdChangedEvent`, `ProposalDurationChangedEvent`, and `GovernanceActionStatusChangedEvent`;
- authority events: `OperatorNominatedEvent`, `OperatorTransferAcceptedEvent`, `OperatorTransferCancelledEvent`, `GovernanceAuthorityChangedEvent`, `UpgradeAuthorityChangedEvent`, and `DirectAuthorityModeChangedEvent`;
- fee and upgrade events: `FeeCollectedEvent`, `CommentsFeeLevelChangedEvent`, `ArtifactFeeLevelChangedEvent`, `ManagedUpgradeCapRegisteredEvent`, `ManagedUpgradeAuthorizedEvent`, and `ManagedUpgradeCommittedEvent`.

14.4 Prompt Registry Events

Prompt registry events are:

- `PromptRegistryCreatedEvent`
- `PromptRegisteredEvent`

14.5 Memory Registry Events

Memory registry events are:

- `MemoryRegistryCreatedEvent`
- `ProviderPolicySetEvent`
- `MemoryEntryCreatedEvent`
- `MemoryEntryPointerUpdatedEvent`
- `MemoryEntryDeletedEvent`
- `MemoryEntryAvailabilitySetEvent`
- `MemoryEntryVersionPolicySetEvent`

15 Canonical Indexing Rules

Indexers are part of the protocol integration surface. They **MUST** distinguish accepted events from rejected events.

15.1 Canonical Acceptance

An event **SHOULD** update canonical domain state only if:

1. its package ID matches the current package ID for the relevant module in the active manifest;
2. the event's registry/root/vault/fee manager fields match the active manifest or an official object reachable from it;
3. for artifact events, the series object belongs to the official publishing path and its artifact type is supported;
4. for comment events, the tree ID matches the series's official comments tree ID;
5. for like events, the likes book ID matches the series's official likes book ID;
6. for governance events, the proposal belongs to the official governance config and registry;
7. for prompt events, the prompt registry ID, root ID, and governance vault ID match the active manifest;
8. for memory events, the memory registry ID, root ID, governance vault ID, entry registry ID, and active-entry mapping match the active manifest and object state.

Events from historical packages **MAY** be retained in a non-current historical table, but frontends **MUST NOT** report current state as complete if they only query an old package or omit active package filters.

15.2 Derived API Views

Indexer APIs MAY expose derived views such as artifact summaries, type lists, search results, lookup responses, official content caches, and wallet activity views. These views are integration products, not new protocol truth. A derived view SHOULD include enough source material for clients to explain how it was constructed: artifact code, series ID, latest version ID, content hash, Walrus blob references, timestamp, accepted package, and cache or refresh time where relevant.

An indexer that exposes derived views MUST continue to apply the canonical acceptance rules above. It MUST NOT promote stale package events, rejected events, failed Walrus reads, or application display policy into protocol facts. When a derived view is incomplete because of provider failure, cache warmup, object hydration failure, or Walrus unavailability, it SHOULD report degraded status rather than silently returning an empty state.

For controller-aware views, the indexer SHOULD resolve and expose the current controller NFT owner, controller NFT ID, control-record ID, and any relevant compatibility mirrors separately. A client MUST NOT infer canonical artifact control from a stale mirrored owner field when controller-object state is available.

15.3 Empty State vs Failed Query

Applications MUST distinguish:

- no canonical events found;
- provider failed;
- provider page limit reached;
- drift check failed;
- old-package events exist but current-package events do not;
- object read succeeded but dynamic child content failed.

Showing “no votes”, “no comments”, or “no artifacts” when the provider failed is a protocol integration bug.

15.4 Protocol Facts vs Interface Policy

An official or third-party interface MAY hide, delist, de-rank, label, refuse to preview, or decline to index an artifact, comment, link, or Walrus reference. Such interface policy MUST NOT be represented as deletion of underlying protocol facts. Sui records, third-party indexer records, and decentralized-storage content may remain retrievable even when one interface does not display them.

Likewise, a visible artifact MUST NOT be interpreted as endorsed, reviewed, or certified by the interface operator. PaperProof establishes typed commitments and official object relationships. It does not establish content truth, legal compliance, academic merit, safety, or social approval.

Independent applications, indexers, wallets, dashboards, agents, and direct chain clients MAY reconstruct protocol views from official objects and canonical events. They MAY apply their own display and moderation policies, but they SHOULD clearly distinguish those application-layer choices from contract-level status and storage availability.

16 Validation Rules

Table 14 summarizes the public validation limits in the publishing and comments modules.

Table 14: Validation limits.

Limit	Value
MAX_VERSIONS_PER_SERIES	168
MAX_METADATA_ATTRIBUTES	4
MAX_METADATA_KEY_BYTES	64
MAX_METADATA_VALUE_BYTES	511
MAX_KEYWORDS	10
MAX_AUTHORS	20
MAX_TAGS	20
MAX_TITLE_BYTES	256
MAX_LONG_TEXT_BYTES	4096
MAX_MEDIUM_TEXT_BYTES	1024
MAX_SHORT_TEXT_BYTES	256
MAX_VECTOR_ITEM_BYTES	128
MAX_CONTENT_HASH_BYTES	128
MAX_WALRUS_BLOB_ID_BYTES	128
MAX_WALRUS_BLOB_OBJECT_ID_BYTES	128
MAX_CONTENT_TYPE_BYTES	64
DEFAULT_MAX_ONCHAIN_COMMENT_BYTES	512
DEFAULT_MAX_COMMENT_DEPTH	64

17 SDK-Facing Contracts

SDKs are not protocol truth, but they SHOULD encode the following rules:

- load deployment manifests without hardcoding only mainnet;
- default to mainnet unless the user selects another network;
- validate Sui addresses, object IDs, package IDs, artifact types, content fields, metadata attributes, and string lengths before building transactions;
- expose build-only, dry-run, dev-inspect, sign-and-execute, and typed-result helpers where the underlying platform supports them;
- return typed IDs extracted from transaction results, including series ID, version ID, comments tree ID, likes book ID, comment ID, proposal ID, and digest;
- expose prompt registry builders and prompt resolution helpers that support latest and pinned version policies;
- expose memory registry builders for creating, deleting, disabling, and governing memory entries without treating private memory bodies as chain data;
- expose query providers and watch APIs that apply canonical filters;
- preserve cause chains in errors and avoid treating provider failure as empty protocol state;
- expose controller-resolution, controller-transfer, and authority-check helpers that distinguish control rights from content license fields and historical authorship;

- expose Walrus content helper functions for publish, read, verify, extend, and owned-blob transfer while preserving Walrus ownership and shared-blob semantics.

For static frontend deployments, browser access to third-party model endpoints, Walrus gateways, and memory relayers depends on ordinary web network policy such as CORS. This is not a Move protocol rule. A local development server may proxy MemWal relayer calls for demos, while a production static site requires a browser-compatible relayer or accepts graceful degradation of memory save/recall without breaking ordinary artifact, prompt, and governance reads.

SDKs SHOULD also support transaction composition where it preserves contract semantics. Compatible Move calls MAY be grouped into a Sui Programmable Transaction Block so a user signs one coherent artifact action. Integrators MUST still respect storage-provider phase boundaries: for example, Walrus reserve and certify operations may require distinct stages even when later certification, publication, add-version, or registry-update calls can be composed.

17.1 Reference Indexer API Expectations

The reference indexer MAY provide server-assisted endpoints for the official application, including Explore summary, type-scoped item lists, search, lookup, official Docs/Blog/Forum content, and compatibility routes for older clients. Those endpoints SHOULD be understood as SDK-facing integration surfaces. They MUST preserve the same canonical filtering rules as persistent indexer tables.

A reference API MAY expose endpoints such as health, JSON metrics, Prometheus metrics, artifact exploration, artifact search, artifact detail, versions, comments, activity, governance proposals, wallet-owned artifacts, wallet votes, analytics summaries, and deterministic airdrop snapshots. These endpoints MUST be documented as derived read APIs. They MUST NOT be described as additional Move entrypoints or as replacements for Sui object state, Walrus content references, deployment manifests, or canonical event acceptance.

Official public content caches SHOULD be keyed by the official manifest entry, series ID, selected version ID, content hash, and Walrus blob reference. A cache hit MAY return verified markdown, package metadata, or rendered-safe content without repeating the full Walrus fetch path. A cache miss or event-driven refresh MUST re-check the chain object or normalized latest version and verify the content hash before presenting content as verified. For Docs/Blog/Forum entries, event-driven refresh is preferred; periodic reconciliation is only a fallback for missed events, manifest drift, service restart, or provider transient failure.

18 Upgrade and Compatibility

PaperProof is upgradeable during its early protocol phase. Package IDs may change. Object IDs for root, registry, vault, and fee manager may remain stable across some upgrades but MUST NOT be assumed without a manifest and drift check.

If only package bindings change and entry signatures/event schemas remain compatible, a manifest update may be enough for many clients. If entry signatures, object layouts, event schemas, or semantic rules change, SDKs and indexers SHOULD publish new versions. Indexers SHOULD store package history so they can explain historical events without mixing them into current canonical state.

19 Security Boundaries

PaperProof security depends on explicit object binding. The following rules are critical:

- The official root is the trust entry point.

- A matching Move type is insufficient to prove an object is official.
- Comments and likes are official only through the series-bound tree and book.
- Governance events are official only through the active governance config and vault.
- Fee settings are official only through the root-bound fee manager.
- Prompt registrations are official only through the manifest-bound prompt registry and referenced PaperProof artifact series.
- Memory entries are official only through the manifest-bound memory registry and usable only when provider, availability, owner, and deletion checks pass.
- Large content remains outside Move; on-chain records commit to references and hashes, not to bytes.
- Private Copilot memory bodies are not stored in PaperProof Move objects.
- PPRF likes and votes are participation signals; they do not prove merit or truth.
- AI agents and frontends may explain and prepare transactions, but wallets or signers remain the authority boundary.

20 Implementation Evidence and Verification Surfaces

This yellow paper is normative only where it describes contract and integration rules. The following figures are included as implementation evidence for the current official deployment and developer workflow. They are not themselves protocol authority.

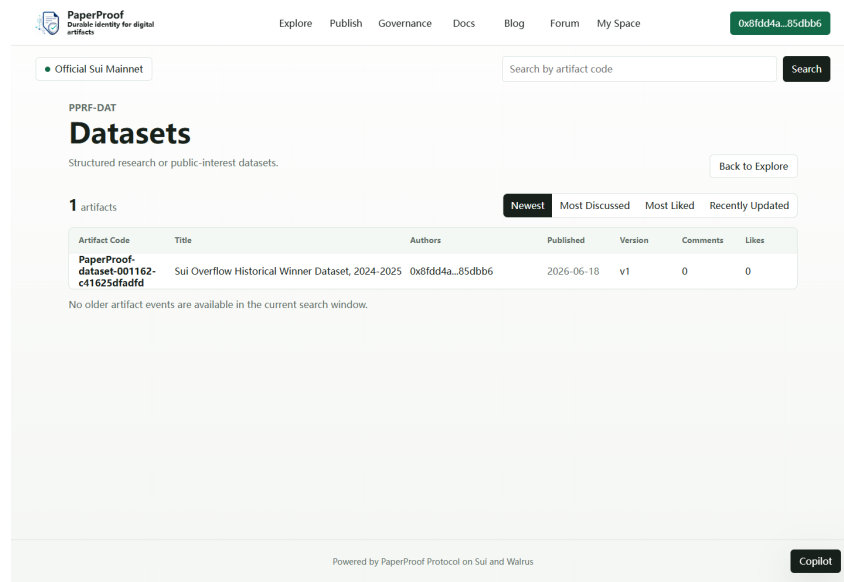


Figure 3: A reference application view derived from canonical artifact events and indexer APIs. Derived views must remain explainable in terms of protocol objects and accepted events.

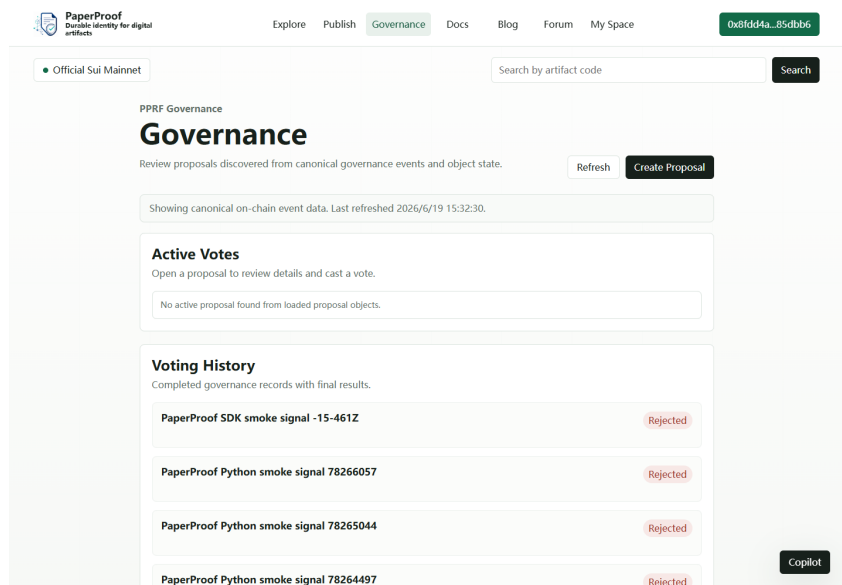


Figure 4: Governance views expose active proposals and voting records, but client display policy must remain distinct from contract-level proposal and voting state.

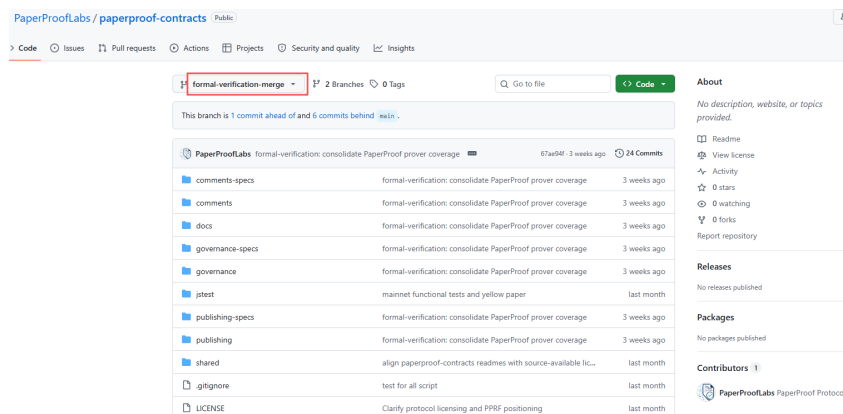


Figure 5: The contracts repository maintains a formal-verification-oriented branch/view alongside implementation sources. Formal artifacts are evidence and review aids; active deployment identity still comes from the manifest and on-chain object bindings.

A Mainnet Deployment Snapshot

The current mainnet manifest records:

Table 15: Mainnet package and object bindings.

Name	Value
network	mainnet
protocolVersion	publishing-v3-governance-v2-comments-v2
pprf package	0x5d2ec9829a9e116de7c2008281a90b96690beb2252af120ad05a25 fe13fae0da
publishing package	0xc9a75e4514db2a37df6f95b4e2b329c065ac6089953bd2c1c0a0c3 89835bd3d8
comments package	0xaeef346fc40bf20af62f4bbbc1608ba2272e80e4ba3d716634026ba a589e9aeba
governance package	0xc1ced3b8ae5281eeeb8cdb5527978e294c54f14a7fd8d65e7e9502 d4ffffb87e
prompt registry package	0x10b9c6e90a896dc3244d047e32724d80de0dc697b5ea12c5fdd892 5131ed4c59
memory registry package	0xbe9527ee927c4a6dcb91d5503758cd731d311813cd70d93914f2bf 58a36db3d1
root	0x7dc6c78b276825499a2204b060394e80b81196eb1f77d2036b503a 2cca15dd78
typeRegistry	0x966ffa24d0a96b34267b62c628f39c830afc9de25438b6502835fa 8a3815d6b5
feeManager	0x7bb8360ea1fa50f923628c929b8726b00eb8968c6a678acde71f97 ae146e9249
governanceVault	0x0df35aa53ef37f8ca8f6a6280d743effa6e0bfc613c5c6c0a78318 ad4a38f875
governanceConfig	0x7ed018db6b2cd7c32692a1c33543fb90d9c36add1226f93cbeb2a8 fb10955dfa
promptRegistry	0x14ec45eb83bb1b0eb22c7e885c7c71ea05b1e22dd05e3e1107dcef 528600b0da
memoryRegistry	0x9a5beeb6610b33c06771c4152c039314784437e802e200afd2ce80 fb88bdf9e2
clock	0x6
PPRF coin type	0x5d2ec9829a9e116de7c2008281a90b96690beb2252af120ad05a25 fe13fae0da::pprf::PPRF

B Abort Code Summary

B.1 Publishing Abort Codes

Name	Code
E_PAUSED	1
E_UNSUPPORTED_ROOT_VERSION	2
E_UNSUPPORTED_REGISTRY_VERSION	3
E_UNSUPPORTED_SERIES_VERSION	4
E_INVALID_ARTIFACT_TYPE	5

Name	Code
E_ARTIFACT_TYPE_DISABLED	6
E_INVALID_INDEX	7
E_INVALID_GOVERNANCE_VAULT	8
E_INVALID_FEE_MANAGER	9
E_NOT_OWNER	21
E_INVALID_STATUS	22
E_INVALID_COMMENTS_TREE	23
E_INVALID_GOVERNANCE_ACTION	24
E_TOO_MANY_VERSIONS	25
E_TOO_MANY_METADATA_ATTRIBUTES	26
E_EMPTY_METADATA_KEY	27
E_METADATA_TEXT_TOO_LONG	28
E_DUPLICATE_METADATA_KEY	29

B.2 Validation Abort Codes

Name	Code
E_EMPTY_CONTENT_HASH	10
E_EMPTY_BLOB_ID	11
E_EMPTY_BLOB_OBJECT_ID	12
E_EMPTY_CONTENT_TYPE	13
E_EMPTY_TITLE	14
E_EMPTY_AUTHOR_LIST	17
E_TOO_MANY_KEYWORDS	18
E_TOO_MANY_AUTHORS	19
E_TOO_MANY_TAGS	20
E_TEXT_TOO_LONG	21
E_EMPTY_TEXT	22
E_EMPTY_VECTOR_ITEM	23

B.3 Comments Abort Codes

Name	Code
E_EMPTY_TARGET_KEY	1
E_TREE_NOT_OPEN	2
E_PARENT_NOT_FOUND	3
E_EMPTY_ONCHAIN_CONTENT	4
E_ONCHAIN_CONTENT_TOO_LARGE	5
E_EMPTY_BLOB_ID	6
E_EMPTY_BLOB_DIGEST	7
E_NOT_TREE_OWNER	8
E_INVALID_TREE_STATUS	9
E_INVALID_COMMENT_STATUS	10
E_COMMENT_NOT_FOUND	11
E_NOT_COMMENT_OWNER_OR_TREE_OWNER	12
E_COMMENT_DEPTH_LIMIT	13
E_INVALID_GOVERNANCE_VAULT	14

Name	Code
E_INVALID_NEW_OWNER	15
E_INSUFFICIENT_PPRF_LIKE_BALANCE	16
E_ALREADY_LIKED	17
E_NOT_LIKED	18
E_UNSUPPORTED_TREE_VERSION	19
E_PARENT_NOT_ACTIVE	20
E_BLOB_ID_TOO_LONG	21
E_BLOB_DIGEST_TOO_LONG	22
E_CONTENT_PREVIEW_TOO_LONG	23
E_INVALID_TREE_FACTORY_CAP	24
E_ROOT_COMMENT_IMMUTABLE	25
E_DELETED_COMMENT_FINAL	26

B.4 Prompt Registry Abort Codes

Name	Code
E_INVALID_GOVERNANCE	1
E_EMPTY_APP_ID	2
E_EMPTY_ROUTE_ID	3
E_INVALID_VERSION_POLICY	4
E_PROMPT_NOT_FOUND	5
E_ZERO_ROOT	6
E_TEXT_TOO_LONG	7

B.5 Memory Registry Abort Codes

Name	Code
E_INVALID_GOVERNANCE	1
E_EMPTY_APP_ID	2
E_EMPTY_PROVIDER	3
E_EMPTY_NAMESPACE_ROOT	4
E_TEXT_TOO_LONG	5
E_ZERO_ACCOUNT	6
E_EMPTY_MEMORY_ID	7
E_EMPTY_ARTIFACT_CODE	8
E_INVALID_VERSION_POLICY	9
E_NOT_OWNER	10
E_ZERO_SERIES	11
E_PROVIDER_NOT_FOUND	12
E_ACTIVE_ENTRY_EXISTS	13

B.6 Governance Voting Abort Codes

Name	Code
E_ZERO_TOTAL_SUPPLY	1
E_PROPOSAL_CREATION_PAUSED	2
E_INVALID_PROPOSAL_TYPE	3
E_INVALID_ACTION_TYPE	4
E_PROPOSAL_NOT_ACTIVE	5
E_ALREADY_VOTED	6
E_VOTING_NOT_ENDED	7
E_PROPOSAL_ALREADY_FINALIZED	8
E_PROPOSAL_NOT_PASSED	9
E_PROPOSAL_NOT_EXECUTABLE	10
E_PROPOSAL_ALREADY_EXECUTED	11
E_INVALID_BOOLEAN_PAYLOAD	12
E_INVALID_VAULT_REGISTRY	13
E_VOTING_ALREADY_ENDED	14
E_EXECUTABLE_ACTION_NOT_ALLOWED	15
E_SIGNAL_ACTION_NOT_ALLOWED	16
E_ACTIVE_PROPOSAL_EXISTS	17
E_PROPOSAL_NOT_FINALIZED	18
E_NO_VOTE_TO_CLAIM	19
E_VOTING_POWER_BELOW_MINIMUM	20
E_PROPOSER_STAKE_BELOW_THRESHOLD	21
E_INVALID_PROPOSER_THRESHOLD	22
E_UNSUPPORTED_CONFIG_VERSION	23
E_UNSUPPORTED_PROPOSAL_VERSION	24
E_INVALID_PROPOSAL_DURATION_EPOCHS	25
E_PROPOSAL_EXECUTION_NOT_EXPIRED	26
E_PROPOSAL_OUTCOME_NOT_YET_DETERMINABLE	27
E_ACTION_NOT_ENABLED	28
E_INVALID_ACTION_ENABLE_TARGET	29
E_NOT_GOVERNANCE_CONFIG_INITIALIZER	30
E_INVALID_PROPOSAL_CONFIG_BINDING	31
E_EMPTY_PROPOSAL_TITLE	32
E_PROPOSAL_TEXT_TOO_LONG	33

C References

- PaperProof contracts repository, Move source modules listed in Section 1.
- PaperProof deployment manifest, docs/deployments/mainnet.json.
- Sui documentation: <https://docs.sui.io/>.
- Walrus documentation: <https://docs.wal.app/>.